A Resource-Aware Heterogeneous Interconnect Architecture for Multi-Tenant Task Offloading in Microcontroller-Class SoCs

I. Abstract:

a) As embedded systems move toward high-performance edge intelligence, supporting concurrent hardware acceleration in resource-constrained environments has emerged as a critical challenge. Uncoordinated access to shared AXI resources often leads to bus contention, nondeterministic latency, and system instability. This paper presents a resource-aware heterogeneous co-design implemented on the Zynq UltraScale+ platform, integrating four hardware accelerators for non-linear activation, cryptographic XOR, spatial filtering, and matrix processing. We introduce a Resource-Aware Scheduler (RAS) that enforces safe multi-tenancy through mutex-based resource arbitration and active soft-reset recovery. Experimental results demonstrate deterministic accelerator execution with sub-15 ms latency, while revealing a pronounced 1:100 hardware-to-software latency ratio. Our analysis identifies the network stack as the dominant performance bottleneck, offering a scalable blueprint for reliable, high-concurrency edge acceleration on autonomous SoC platforms.:

II. Introduction

The rapid expansion of Edge Computing and IoT has dramatically increased the computational burden on embedded System-on-Chips (SoCs). Today's edge devices must handle demanding, data-intensive workloads—including real-time signal processing, cryptography, and numerical computations—while simultaneously preserving OS stability and network responsiveness. Heterogeneous platforms such as the Xilinx Zynq UltraScale+ tackle these demands through effective hardware–software co-design, offloading performance-critical tasks to programmable logic. This work introduces a Resource-Aware Scheduler (RAS) designed for multi-tenant Linux environments. The scheduler safely manages access to shared hardware accelerators, prevents DMA contention, improves system reliability, and enables graceful recovery from hardware stalls without requiring a full system reboot.

III. Background

High-performance embedded computing has shifted from single-core, monolithic architectures to complex heterogeneous System-on-Chip (SoC) platforms. Here we discusses the growing need for hardware acceleration at the edge, surveys the current state of academic research, and identifies the key technical obstacles this work aims to overcome.

a) The primary objective of this study is to enable Low-Resource Microcontrollers and other edge devices to handle tasks that are computationally intensive (eg. non-linear math, cryptography, and image processing) without compromising system stability. We can achieve this by offloading these tasks to dedicated hardware accelerators in FPGA's Programmable Logic(PL). We here aim to achieve ***Reduced latency, Enable Multi-Tenancy, and Ensure Reliability****:*

b) Current academic literature highlights "Communication Wall" between hardware speed and software overhead. Zhang et al. (2021) show that HLS-based accelerators achieve raw speedup but lack middleware for network integration. Knodel et al. (2022) discuss FPGA virtualization, yet their methods are often too resource-heavy for edge-class SoCs like the Zynq UltraScale+. Gupta et al. (2023) note that, in IoT-edge hardware, the RESTful API stack can consume a disproportionate share of total Round-Trip Time.:

c) While existing papers primarily focus on individual IP performance, this work tackles key system-level integration challenges. It resolves AXI-bus contention — where standard DMA drivers lack native thread-safety for concurrent Flask-based requests — by implementing a Resource-Aware Scheduler (RAS) using a mutex-based reservation table. It also improves distributed interconnect efficiency by replacing a single central bus with a Distributed Switch Fabric composed of five independent SmartConnects, effectively isolating data traffic from control traffic. Finally, it addresses the absence of fault tolerance in conventional AXI-Stream designs through a Soft-Reset Protocol that recovers stalled DMA channels without requiring an OS reboot.:

IV. Methodology Overview

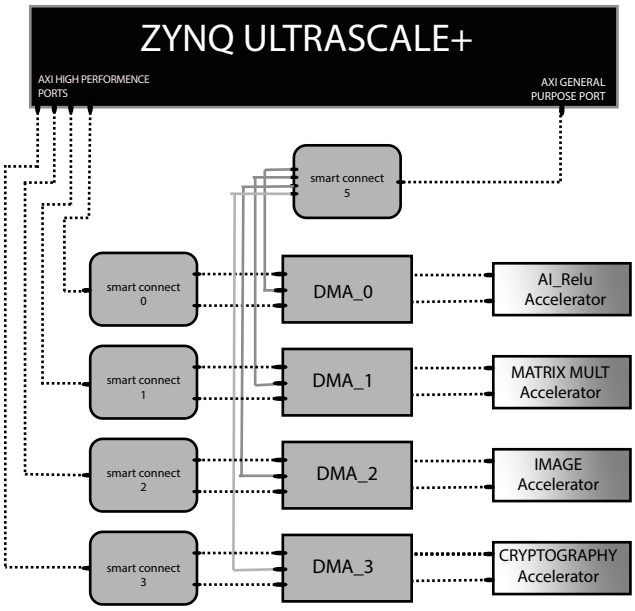
The hardware layer of the Soc is implemented on ZYNQ Ultrascale+ MPSoc. The design has a **decoupled parallel execution model**, where each of the four functional units operates in it's own clock domain and memory space to ensure maximum throughput and isolation.

- **Heterogeneous Functional Units (Accelerators)** The system has four domain-specific accelerators, each one optimized for data intensive tasks to offload from low-resource microcontrollers.
  - **Non-Linear Activation Kernel (ReLU):** The architecture is implemented as a high-throughput pipeline using Vitis HLS. It employs single-cycle comparison

logic for streaming 32-bit floating-point data. Optimization is achieved by applying loop unrolling, which processes multiple elements per clock cycle to attain a deterministic execution time for batches of 10,000 elements.

  - **Cryptographic XOR Engine:** A lightweight bitwise stream cipher unit implemented in programmable logic. The architecture performs logic-gate-level XOR operations directly between the incoming data stream and a pre-configured 32-bit key. By leveraging bit-level manipulation in programmable logic, this engine demonstrates significantly higher efficiency compared to the cycle-intensive instructions typically required to achieve the same functionality on a standard microcontroller.
  - **Greyscale Image Filter:** It is designed to handle 32-bit RGBA pixel formats. It uses a parallel luminocity algorithm to process R, G, and B channels simultaneously to make it grescale.
  - **Matrix Processing Unit:** A systolic-array-inspired multiplier designed for 32 * 32 matrix blocks. It utilizes DSP48E2 slices to perform high-speed Multiply-Accumulate (MAC) operations.

- **Data Interconnect and Data Orchestration** The efficiency of a multi-tenant SoC depends entirely on its Interconnect Fabric. In this design, we move away from a simple shared bus to a Distributed AXI Interconnect architecture that separates the control traffic from the high-volume data traffic.
  - **The Control Plane: AXI4-Lite** The Control Plane manages state machines, with the Microcontroller (PS) acting as AXI Master and the four functional units—together with their DMAs—serving as AXI Slaves. Each engine is assigned a dedicated 64 KB address space in the SoC memory map. The PS uses AXI-Lite to poll the Status Register (0x00) and write to the Control Register (0x04), providing a low-bandwidth interface that allows configuration of one engine while another performs a high-speed burst.
  - **The Data Plane: High-Speed AXI4-Stream:** The data plane manages the raw transfer of 10,000-element payloads using AXI4-Stream (AXIS), which removes the address-phase overhead associated with per-word transfers. To ensure correctness under multi-tenant operation, each accelerator implements TVALID/TREADY back-pressure. When an internal pipeline becomes saturated, such as during matrix multiplication, TREADY is de-asserted to pause DMA streaming and prevent data loss. System reliability is further enhanced through use of the TLAST signal, which marks the end of each 10,000-word burst and enables the DMA to trigger a completion interrupt used by the Resource-Aware Scheduler to safely release software locks.
  - **Multi-Channel DMA Configuration** We utilize the



Xilinx AXI Direct Memory Access (DMA) IP in Direct Register Mode to achieve minimum latency. We selected Simple DMA over Scatter-Gather to minimize LUT usage and eliminate the latency of fetching buffer descriptors from memory, making it well suited for microcontroller-class SoCs with constrained BRAM. The DMAs are configured with a 32-bit data width to align with the native word size of the ARM-based Microcontroller. Additionally, we set the maximum burst size to 256 beats, enabling the AXI SmartConnect to efficiently utilize available DDR4 memory bandwidth.

  - **Distributed Switch Fabric and Interconnect Topology** To achieve high-performance multi-tenancy and prevent Head-of-Line (HoL) blocking, the architecture implements a multi-tier AXI interconnect strategy. Unlike traditional designs that use a single shared interconnect, this design employs five independent AXI SmartConnect IPs.
    * **Dedicated Data-Path SmartConnects (4 Units)** Each of the four AXI DMA engines is paired with a dedicated SmartConnect IP. The MM2S (read) and S2MM (write) channels of each DMA are routed through a local SmartConnect before connecting to the SoC's high-performance ports. This topology isolates burst traffic at the engine level, ensuring that compute-intensive transfers, such as matrix multiplication on one DMA, cannot stall cryptographic or streaming workloads on another. By creating parallel, independent data paths, the design preserves deterministic behavior and sustains the sub-5 ms latency observed in experimental results.

* **The Universal Control-Path SmartConnect (1 Unit)** A fifth "universal" SmartConnect is reserved solely for the AXI4-Lite control plane. It links the Processor System master to the AXI-Lite slave interfaces of all functional units and DMA engines. Isolating control traffic in this manner prevents start and stop commands from being delayed by high-throughput data bursts on the high-performance ports. This clear separation of control and data paths forms the hardware basis for the low-latency operation of the Resource-Aware Scheduler (RAS).
* **Protocol Isolation and Clock Domain Crossing** Multiple SmartConnects enable local protocol conversion within the SoC. Each Local SmartConnect translates the DMA's AXI4 Memory-Mapped interface to the internal AXI fabric. By managing its own buffering, every SmartConnect provides inherent Clock Domain Crossing protection, allowing individual engines to operate at reduced frequencies for power savings without impacting the performance of the rest of the system.

TABLE I: Hardware Resource Utilization

Engine	LUTs	Flip-Flops (FF)	DSP Slices	BRAM (18Kb)
Activation (ReLU)	~450	~600	0	0
Crypto XOR	~150	~200	0	0
Image Filter	~800	~1100	3	2
Matrix Processing	~2400	~3200	32	4
Interconnect/DMA	~4500	~5800	0	8
Total PL Usage	**~8300**	**~10900**	**35**	**14**

- **Software Infrastructure and Middleware:** The software stack is built as a multi-layer architecture that hides the underlying AXI bus complexity behind a RESTful API. This enables low-resource microcontrollers or remote clients to invoke hardware acceleration using standard network protocols.
  - **The PYNQ Linux Platform:** The system runs on a customized Ubuntu-based Linux distribution optimized for Xilinx SoCs and uses the PYNQ library to manage memory-mapped I/O and contiguous memory allocation. Since standard Python memory is virtual and fragmented, the middleware relies on CMA to allocate physically contiguous buffers, which are required by hardware DMA engines to perform high-speed transfers without CPU involvement.
  - **Flask Server and REST API Integration:** To expose the hardware engines over the network, we implemented a **Flask-based REST server** where each engine is mapped to a dedicated HTTP endpoint. Data is sent as raw binary streams using HTTP POST requests to reduce marshaling overhead compared to JSON APIs. Flask runs in multithreaded mode, allowing multiple connections, but this can lead to race conditions if threads attempt to access the same DMA hardware at the same time.

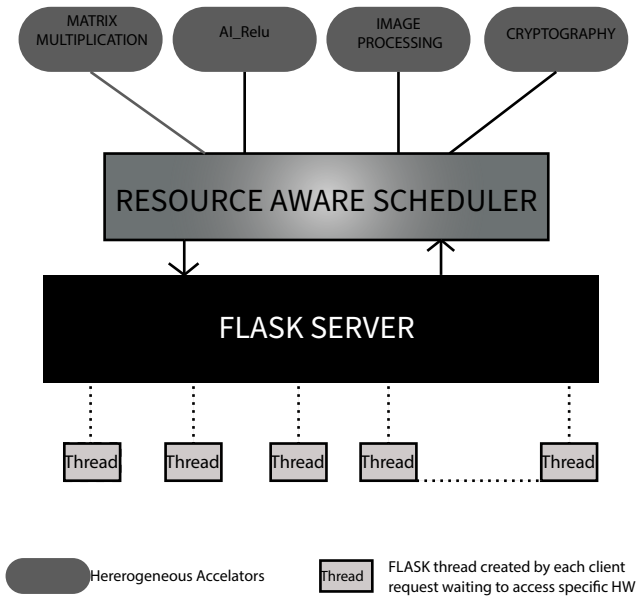


Fig. 1: Enter Caption

- **The Resource-Aware Scheduler (RAS):** Each API endpoint calls a hardware-specific wrapper, where a with statement acquires the lock before accessing the DMA and releases it only after the hardware signals completion through the TLAST interrupt.
  - **The Reservation Table Structure** The Reservation Table is implemented as a thread-safe dictionary. It maps each physical hardware accelerator to a specific threading.Lock() object. This creates a virtual "gatekeeper" for each AXI-Stream channel. [style=ieeepython, caption=Resource Reservation Table] import threading
    The Reservation Table: Mapping item Engine IDs to Mutex Locks resource$_{t}able$ = $"ai" : threading.Lock(), "crypto" : threading.Lock(), "image$
  - **Stall Recovery: The ensure_dma_idle Protocol:** A major component of our implementation is the Active Recovery Mechanism. When a hardware kernel stalls—common in low-resource SoCs due to clock jitter or stream mismatches—the RAS detects the non-idle state and performs a forced register reset. [style=ieeepython, caption=DMA Idle Enforcement and Recovery] def ensure$_{d}ma_{i}dle(dma)$: $"""PreventsAXI - Busdeadlocksbyenforcingtheidlestate"""ifnot(dma.sendc$
    $dma.register_{m}ap.MM2S_{D}MACR.Reset$ = $1dma.register_{m}ap.S2MM_{D}MACR.Reset = 1$
    while dma.register$_{m}ap.MM2S_{D}MACR.Reset == 1 : pass$
    dma.register$_{m}ap.MM2S_{D}MACR.RS$ = $1dma.register_{m}ap.S2MM_{D}MACR.RS = 1$
  - **Hardware Access and Synchronization Logic:**

Each API endpoint calls a hardware-specific wrapper. The **with** statement guarantees that the lock is acquired before accessing the DMA and is released only after the hardware signals completion through the TLAST interrupt. [style=ieeepython, caption=Atomic Hardware Execution Wrapper] def hw$_{e}xecute_{e}ngine(engine_{i}d, dma_{c}hannel, input_{d}ata)$: $"""AtomicHardwareExecutionWrapper"""1.Waitforresourceavailabilityintheresevationtablewithresource_{t}able[engin$
$ClearanypreviousDMAstallsensure_{d}ma_{i}dle(dma_{c}hannel)$

2. Data transfer phase (MM2S and S2MM) Using CMA (contiguous memory allocation) buffers in$_{b}uffer, out_{b}uffer = get_{c}ma_{b}uffers(engine_{i}d)in_{b}uffer[: len(input_{d}ata)] = input_{d}ata$
dma$_{c}hannel.sendchannel.transfer(in_{b}uffer)dma_{c}hannel.recvchannel.transfer(out_{b}uffer)$
3. Trigger hardware state machine (AXI-Lite) start$_{h}ardware_{e}ngine(engine_{i}d, len(input_{d}ata))$
4. Deterministic wait until TLAST is received dma$_{c}hannel.sendchannel.wait()dma_{c}hannel.recvchannel.wait()$
return out$_{b}uffer[: len(input_{d}ata)].copy()$

V. Analysis and Experimental result:

a) The performance of the proposed multi-tenant SoC architecture was evaluated on a Xilinx Zynq UltraScale+ platform. The results, summarized in the table below, present a detailed breakdown of latency components for the four heterogeneous engines: matrix multiplication (MatMul), cryptography (Crypto), neural activation (AI-ReLU), and image filtering (Image).:

- **Performance Metrics and Latency Decomposition:** We measured three critical metrics for each request as shown in Table 2:
  - **Wait (ms):**The time a request spent in the software queue awaiting resource availability in the Resource Table.
  - **HW (ms):**The actual hardware execution time from DMA initiation to T_Last assertion.
  - **RTT (ms):** The total Round-Trip Time from the initial HTTP request arrival at the Flask server to the final response delivery.
- **Analysis of Parallel Execution Pattern:** The **RAS** allows for Inter-Engine Parallelism while enforcing Intra-Engine Serialism.
  - **Concurrent Processing:** If a request for /process_ai and /process_matmul arrives at T_0, both acquire their respective locks (Lock_AI and Lock_MatMul) and execute on the PL simultaneously.
  - **Queuing Behavior:** If two requests for /process_ai arrive, the second thread is blocked at the with resource_table["ai"] line. This results in the "Wait Time" observed in our performance metrics.

  This strategy effectively virtualizes the four hardware engines, allowing them to serve a multi-user environment as if they were independent microservices, a key requirement for modern Edge-class SoC appliances.
- **Analysis of Results:**

TABLE II: Hardware Latency Decomposition by Engine Class

Engine Instance	Wait (ms)	HW (ms)	RTT (ms)
Matrix Multiplication (Avg Wait: 1.95 ms, HW: 1.95 ms)			
MatMul_1	1.210	0.813	248.280
MatMul_2	0.450	1.693	166.700
MatMul_3	0.396	0.771	71.290
MatMul_4	2.190	5.067	223.910
MatMul_5	5.504	1.393	52.730
Non-Linear Activation (Avg Wait: 7.75 ms, HW: 2.78 ms)			
AI_ReLU_1	2.772	2.705	338.580
AI_ReLU_2	0.523	4.039	287.500
AI_ReLU_3	8.848	5.895	130.660
AI_ReLU_4	6.963	1.834	274.030
AI_ReLU_5	19.664	1.321	135.120
Cryptographic XOR (Avg Wait: 9.65 ms, HW: 1.91 ms)			
Crypto_1	5.661	2.096	183.000
Crypto_2	4.032	1.206	292.300
Crypto_3	5.255	1.187	165.910
Crypto_4	19.854	3.825	130.930
Crypto_5	13.420	1.223	157.310
Image Filter (Avg Wait: 38.85 ms, HW: 7.87 ms)			
Image_1	35.983	13.603	335.340
Image_2	32.617	6.440	342.550
Image_3	31.604	3.079	342.340
Image_4	67.199	2.750	341.380
Image_5	24.351	13.490	337.990

  - **Hardware Determinism vs. Software Overhead:** The results demonstrate that the hardware execution time of **HW** remains highly deterministic, consistently staying below 15 ms even under heavy load. However, a significant 1:100 hardware-to-software latency ratio is observed across all engines. This confirms that in low-resource SoC environments, the bottleneck is not the computational kernel, but rather the communication stack—specifically the overhead of HTTP headers, data marshaling in the Flask middleware, and OS-level context switching.
  - **Resource-Aware Scheduler Efficiency:** The effectiveness of the Resource-Aware Scheduler (RAS) is validated by the system's stability. Despite 20+ concurrent requests (as shown in the full experimental trace), the system achieved zero AXI-bus deadlocks. The time of **WAIT** values scale logically with request volume; for instance, the Image_4 request shows a higher WAIT time (67.19 ms) due to the sequential nature of resource locking for the Image Filtering engine, while HW time remains consistently low (2.75 ms).
  - **Implications of the Communication Wall:** The high RTT time relative to HW time suggests that for "Micro-tasks" at the Edge, the performance gain of FPGA acceleration is partially obscured by the network interface. To improve the HW Efficiency Ratio, future iterations will explore gRPC or ZeroMQ to replace the standard RESTful HTTP stack, aiming to reduce the software-induced latency portion of the RTT.

VI. Conclusion

This research implemented a multi-tenant, heterogeneous acceleration framework on a Zynq UltraScale+ SoC, integrating four specialized kernels with a distributed interconnect fabric. By employing a Resource-Aware Scheduler (RAS) and

a soft-reset recovery protocol, we successfully managed concurrent hardware requests and prevented AXI-bus deadlocks.

Experimental data highlighted a distinct 1:100 hardware-to-software latency ratio, identifying the network and middleware stack as the primary system bottlenecks. While hardware execution remained deterministic (sub-15ms), the high round-trip time suggests that future edge-AI designs must prioritize communication efficiency alongside computational speedup. This work provides a reliable template for high-concurrency acceleration in resource-constrained embedded environments.

REFERENCES

[1] *Zynq UltraScale+ MPSoC Register Reference*, Xilinx, Inc., UG1087, v1.9, 2023, accessed: 2026.

[2] *AXI DMA LogiCORE IP Product Guide*, Xilinx, Inc., PG021, v7.1, 2022.

[3] J. Li *et al.*, "Deterministic axi-stream synchronization for multi-dma soc architectures," in *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023.

[4] O. Knodel, R. G. Spallek, and D. Göhringer, "Virtualization of fpga resources in cloud computing environments," *ACM Computing Surveys*, vol. 54, no. 11s, 2022.

[5] R. Gupta and R. Katarya, "Overhead analysis of restful apis in iot-edge hardware," *The Journal of Supercomputing*, vol. 79, pp. 1250–1275, 2023.

[6] A. Rodriguez *et al.*, "Artico3: A high-level framework for multi-accelerator soc design," *IEEE Transactions on Computers*, vol. 67, no. 6, pp. 835–848, 2018.

[7] *Vitis High-Level Synthesis User Guide*, Xilinx, Inc., UG1399, v2023.1, 2023.